Introduction à la conteneurisation

Tic-1

Tek-up

Introduction

- Historiquement, les biens étaient manipulés manuellement: chargés et déchargés individuellement par les travailleurs.
  - Le transport de biens était donc coûteux, lent et peu fiable (biens perdus, abîmés, etc.)
- Les conteneurs physiques: grandes boîtes de métal de taille standardisée qui transportent les biens matériels.





Introduction

- Indépendamment de la nature des biens transportés, le transporteur doit s'assurer que le conteneur dans son ensemble arrive intact.
  - Il peut les empiler, choisir le mode de transport adapté à la destination, etc.
    →optimiser les coûts et la logistique.
- Le client a, de son côté, la garantie que ses biens arriveront dans le même état que lors du chargement.
  - Il n'a pas à se soucier de l'infrastructure de transport.

Déploiement natif

- Installation native des applications sur un même hôte sans machine virtuelle ou conteneur.
- Plusieurs situations problématiques peuvent survenir :
  - Effort d'installation, de configuration et de migration (d'un environnement à un autre)
  - Difficulté de cohabitation et d'interaction
  - Risque de dégradation ou de discontinuité de services

Virtualisation

- La virtualisation utilise un hyperviseur pour émuler le matériel

  → possibilité d'exécuter plusieurs SEs en parallèle.

- Offre un niveau d’isolation élevé.
- Les problèmes évoqués par l’architecture traditionnelle

(non-virtualisée) sont donc résolus, mais d’autres apparaissent:

- Poids lourd de MVs (taille se mesure en GBs)
- Les ressources attribuées à une MV lui sont réservées
- Temps de démarrage important

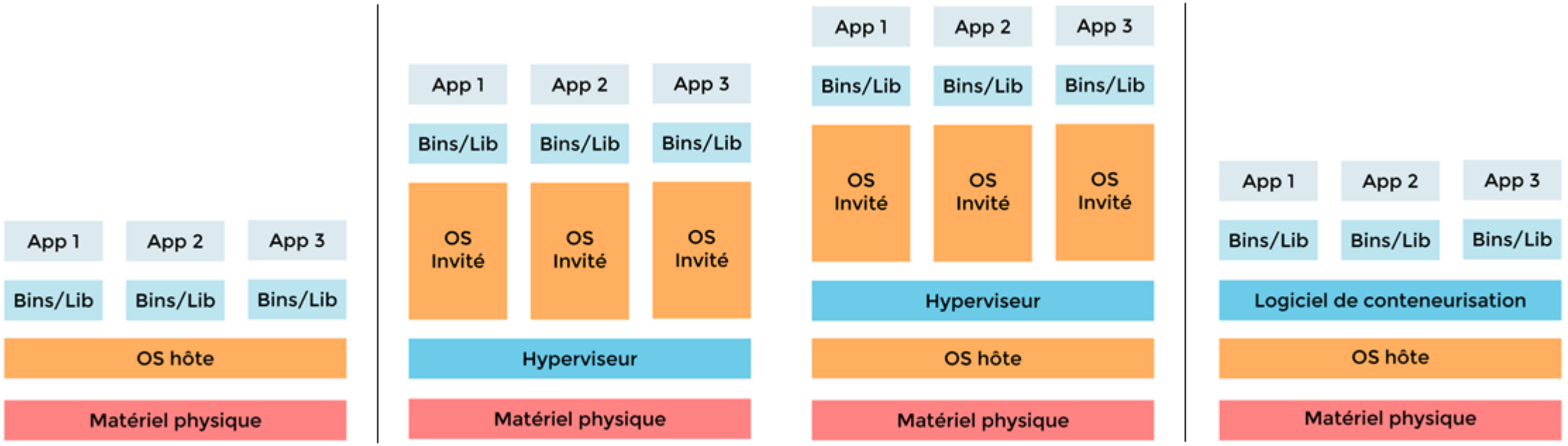
Conteneurisation

- La conteneurisation permet d'isoler / packager / envelopper des applications avec leur environnement d'exécution complet contenant tous les fichiers nécessaires à leur fonctionnement.
- Il s’agit d’une virtualisation du système d’exploitation: les conteneurs partagent le même noyau de SE.

Conteneurisation

- Un conteneur est une unité standardisée qui regroupe un processus ou un ensemble de processus de manière isolée du reste du système.
- Il est plus léger qu'une MV (occupe moins d'espace) et il démarre presque aussi vite que le processus qu'il encapsule.
- Les applications conteneurisées sont plus faciles à déplacer d'un environnement à un autre (développement, test, production, etc.), tout en conservant l'intégralité de leurs fonctions.
- La conteneurisation (les conteneurs) n'exclut pas la virtualisation (les machines virtuelles): les deux techniques peuvent être mixées en fonction des besoins.
    - Une machine virtuelle peut accueillir des conteneurs

Conteneurisation



Avantages

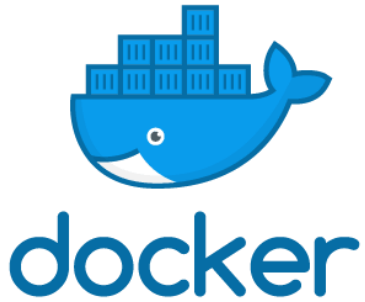
- Virtualisation légère (un conteneur n’a pas de SE et il ne réserve que les ressources nécessaires).
- Mieux développer, déployer et gérer les applications.
- Portabilité des conteneurs sur une large variété d’hôtes
- Même fonctionnent dans les différents environnements (développement, test, production, etc.)
- Permettre le contrôle des versions des applications.
- Séparer les responsabilités des équipes de développement et d'exploitation.
- Solution idéale pour le développement des applications de type "*microservice*".

Inconvénients

- Difficulté de gérer un grand nombre de conteneurs simultanément
- Enjeux de sécurité: protection de l'intégrité des conteneurs, des applications qu'ils renferment et de l'infrastructure sur laquelle ils s'appuient.
- Incompatibilité avec certaines types de tâches

Solutions de conteneurisation

- Docker
- LXC
- rkt (CoreOS Rocket)
- Open VZ
- Etc.







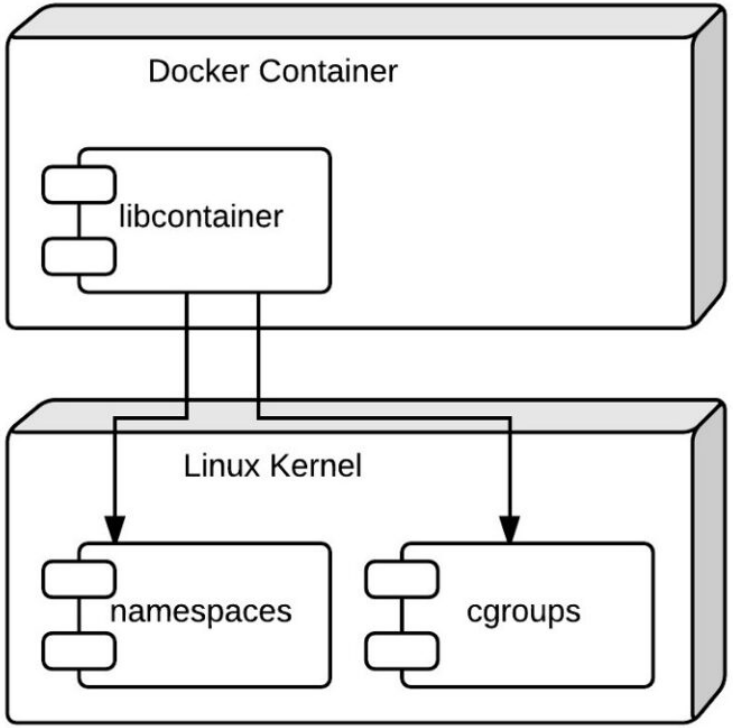


Docker

- Docker est une plateforme logicielle qui permet de créer et déployer des applications rapidement.
- Docker intègre les applications dans des conteneurs qui rassemblent tous les éléments nécessaires à leur fonctionnement dont le code, l'environnement d'exécution, les bibliothèques, etc.
- • Deux modèles de licence:
  - Docker Community Edition (CE): gratuit et accessible à tous.
  - Docker Enterprise Edition (EE): sous forme d'abonnement payant.

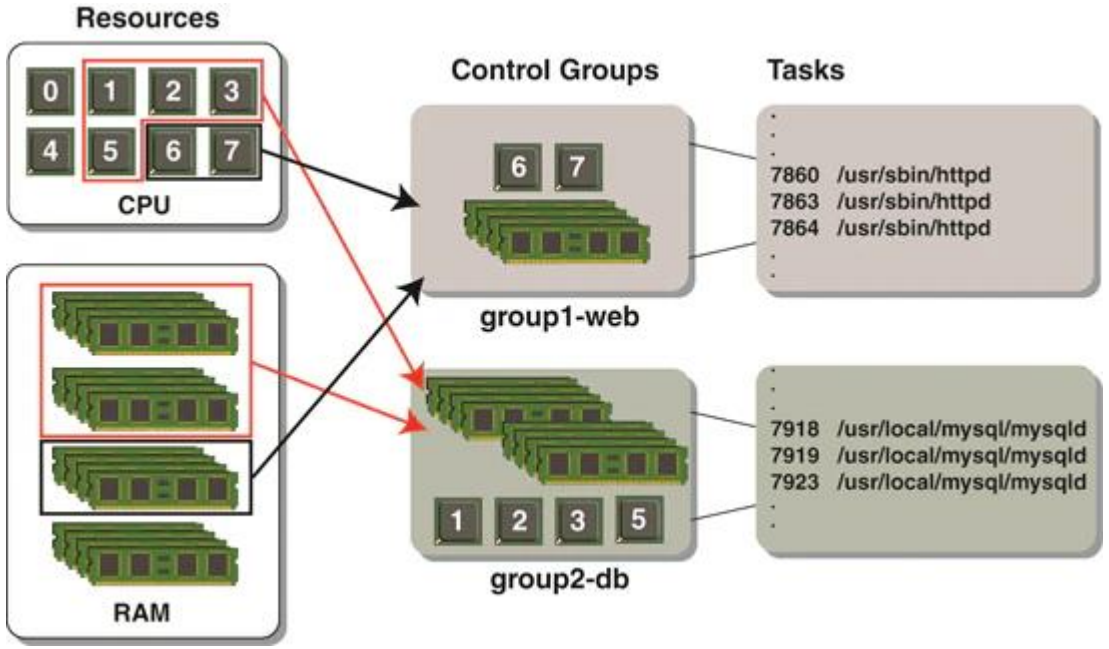
Technologies sous-jacentes

- Initialement, Docker utilisait notamment LXC (Linux Containers), mais par la suite il a développé sa propre bibliothèque de bas niveau nommée **libcontainer**
- Elle encapsule les fonctionnalités fondamentales, proches du noyau du système d'exploitation, dont la combinaison permet la conteneurisation
- Les services Linux de base sur lesquels s'appuie la libcontainer de Docker sont:
  - Cgroups (pour Control Groups)
  - Namespaces



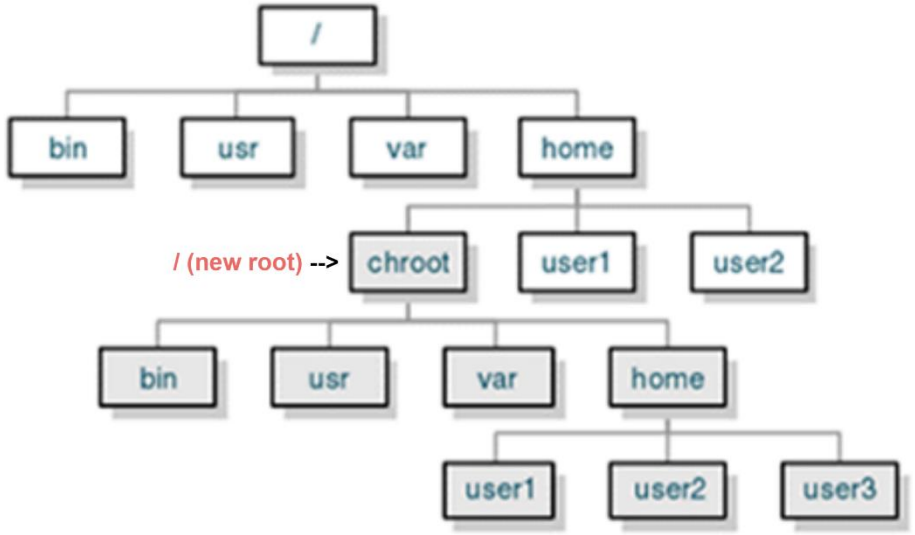
Technologies sous-jacentes

- Le **CGroups** permet de partitionner les ressources d'un hôte et de contrôler la consommation de ces ressources par processus.



Technologies sous-jacentes

- Les **Namespaces** permettent d'isoler les processus de sorte qu'ils ne voient pas les ressources utilisées par les autres. Ils se basent sur la commande **chroot** qui permet de changer le répertoire racine du système pour un processus donné (il a l'impression d'être à la racine du système).



Conteneur Docker

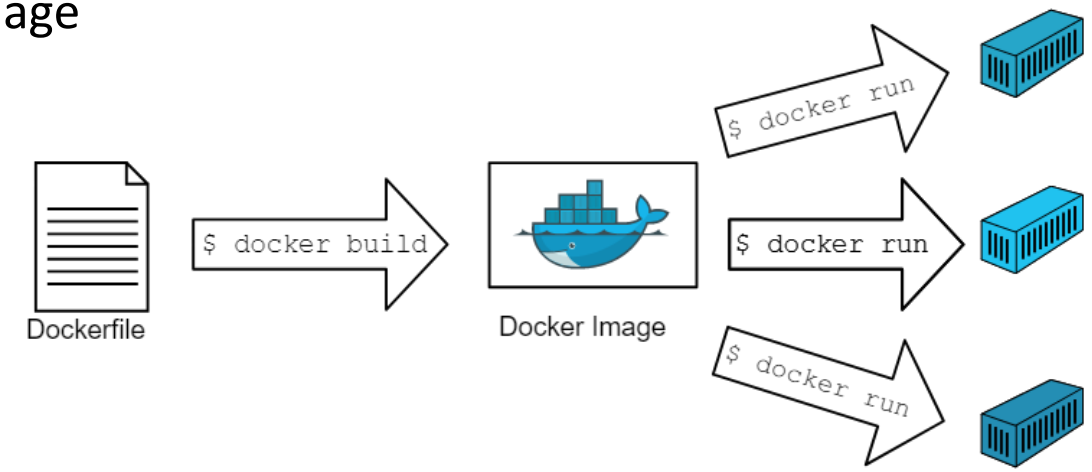
Un conteneur est donc un système de fichiers sur lequel s'exécutent un ou plusieurs processus de manière :

- **contrainte** : grâce à CGroups qui contrôle le partage de ressources
- **isolée** : grâce à Namespaces qui assure que les conteneurs ne se voient pas les uns les autres.

→ Le CGroups gère la distribution des ressources alors que les Namespaces apportent l'isolation nécessaire à la création de conteneurs.

Terminologie

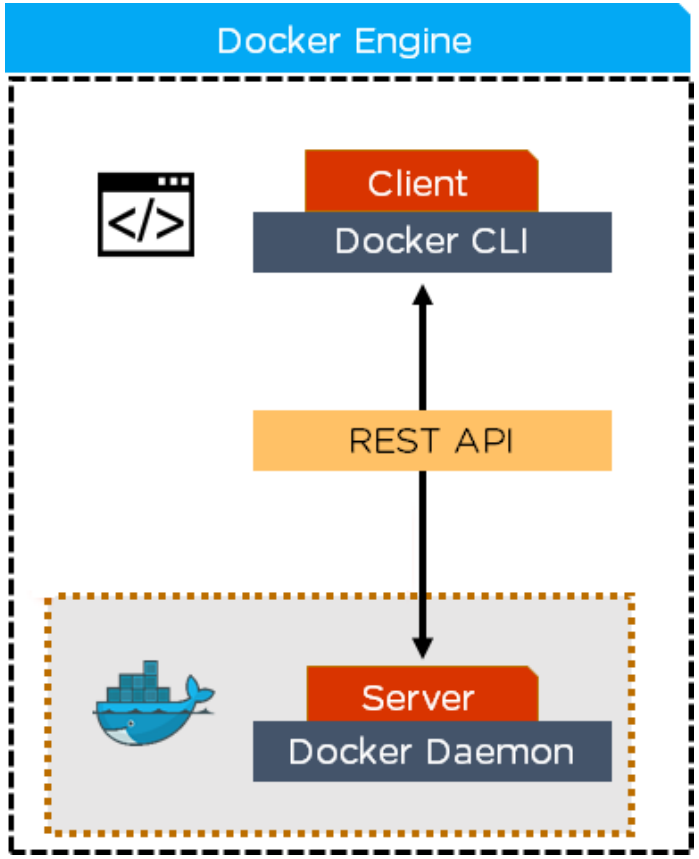
- **Dockerfile**: un fichier texte qui décrit les différentes instructions permettant la création des nouvelles images.
- **Docker image** : un modèle (template/blueprint) en lecture seule qui définit les conteneurs.
- **Docker container**: une instance d'image manipulable. On peut lancer plusieurs conteneurs à partir d'une seule image



Docker Engine

Docker utilise une architecture client/serveur:

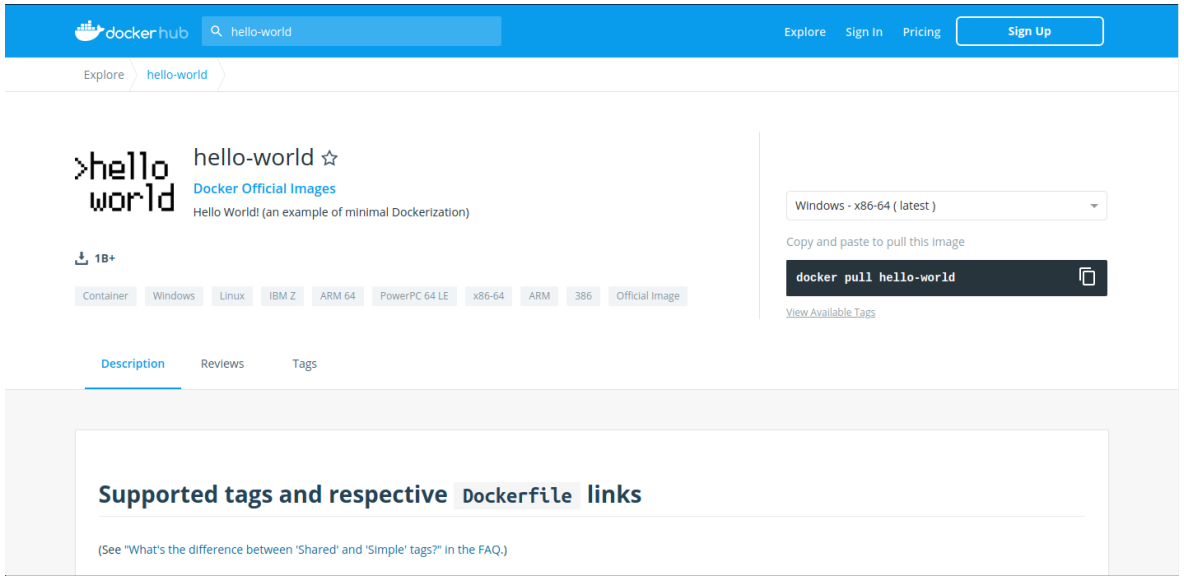
- **Un serveur:** une sorte de long-running programme appelé démon qui est responsable de la gestion des objets docker (images, conteneurs, etc.).
- **Une API REST:** représente l'interface de communication entre le client et le serveur.
- **Un client:** une ligne de commandes (CLI) pour accepter les commandes des utilisateurs



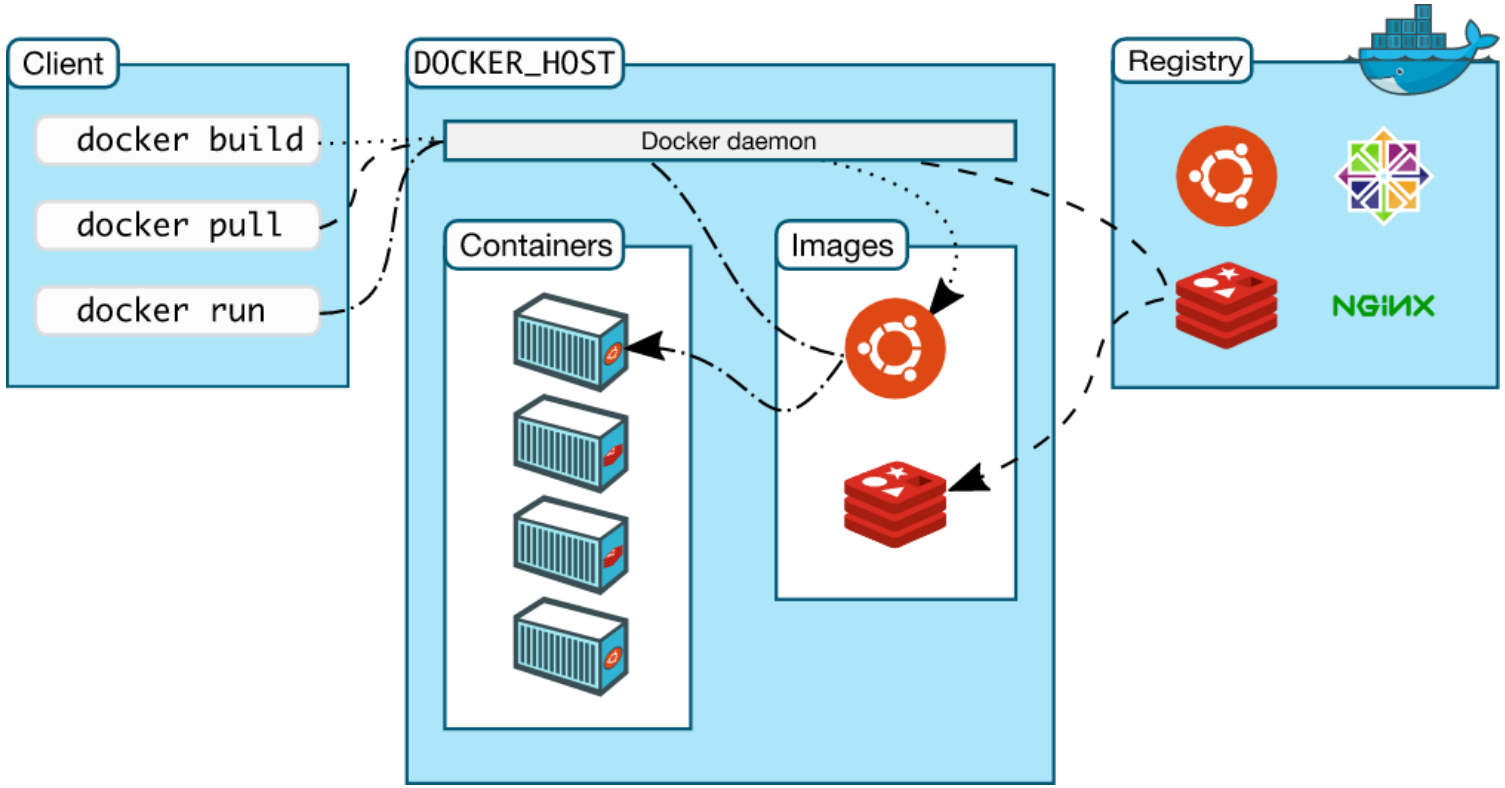
Docker Registry

- Docker Registry est un système de stockage et de distribution des images (exemple: Docker Hub). Une image peut avoir plusieurs versions identifiées par leurs tags.
- Contraintes de sécurité liées à l'utilisation des images publiques (publiées par les autres)





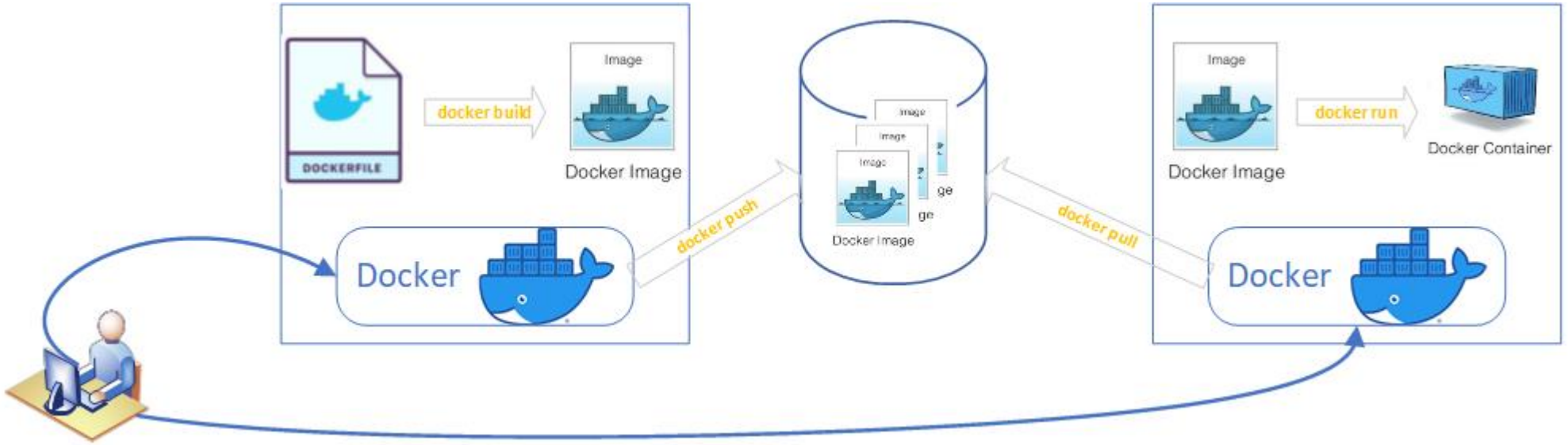
Architecture



Docker Workflow

1. Une fois le fichier Dockerfile (liste des instructions pour la création d'une image) est créé, Docker Client demande à Docker daemon de compiler le fichier Dockerfile en une image avec la commande *docker build*
2. Docker Client demande à Docker daemon de publier l'image dans le registre Docker avec la commande *docker push*
3. Docker Client demande à Docker daemon de créer un conteneur en utilisant une image spécifique avec la commande *docker run*
   - Si l'image souhaitée ne se trouve pas en local, Docker daemon télécharge l'image à partir du registre Docker avec la commande *docker pull*
4. Docker daemon communique avec le noyau Linux via libcontainer pour créer le conteneur

Docker Workflow



Kubernetes

- Taux d'utilisation croissant de conteneurs + infrastructure de plus en plus décentralisée

  → nécessité d'un orchestrateur

- Kubernetes (K8S) est une plateforme open source développée par Google qui permet l'orcherstration et l'automatisation de gestion des conteneurs sur des clusters (différents environnements)
- C'est l'orchestrateur le plus populaire pour la gestion de conteneurs.

  – Autres solutions: Docker Swarm, Apache Mesos



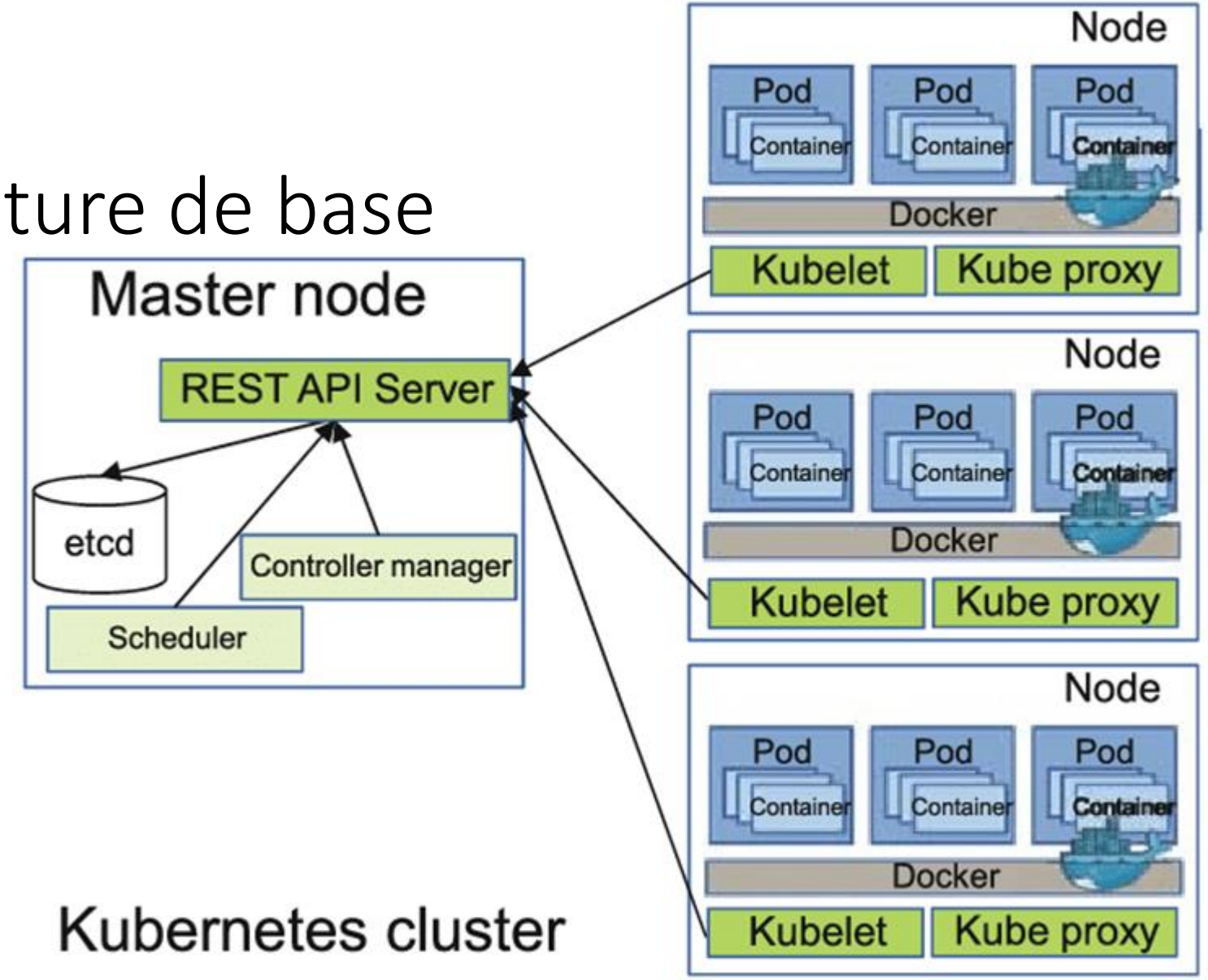
Rôle

- Orchestrer des conteneurs sur plusieurs hôtes
- Planifier les charges de travail dans le cluster
- Vérifier l'intégrité des applications
- Assurer le monitoring

Architecture de base

Architecture

maître / esclave



Architecture de base

- Le maître Kubernetes (Kubernetes master) est un seul nœud du cluster désigné comme nœud maître (master node) qui exécute principalement :
  - **Controller Manager**: supervise l'état du cluster
  - **Scheduler**: répartit les de travail entre les nœuds (selon la disponibilité)
  - **etcd** : enregistre la configuration complète du Cluster
  - **API Server**: gère toutes les communications entre les différents composants
- Il n'a pas besoin de beaucoup de ressources
- C'est le nœud le plus important dans l'architecture

  → deux ou plus nœuds maître de backup

Architecture de base

- Chaque nœud non maître (nœud esclave ou worker node) du cluster est composé de :
  - **Kubelet** : surveille l'état des conteneurs sur le nœud et communique avec le nœud maître
  - **Kube-proxy** : maintient les règles réseau sur les nœuds telle que l'affectation des adresses IP, etc.
  - **Container Runtime** : effectue les opération sur les conteneurs dans un nœud. Exemple : Docker
  - **POD** : l'unité de base de Kubernetes. Un ou plusieurs dans un nœud

→ consomment plus de ressources

POD

- C’est l’unité de base de Kubernetes
- Elle regroupe un seul (recommandé) ou plusieurs conteneurs (généralement étroitement liés)
- Kubernetes fonctionne en interagissant avec les PODs (pas avec les conteneurs directement)
- Chaque POD dans le cluster possède une adresse IP unique
- Un POD peut définir un espace de stockage appelé volume et l’exposer à ses conteneurs afin de :
    - partager de données entre les conteneurs de même POD
    - préserver les données au delà de cycle de vie d’un conteneur
- Les PODs sont hébergés dans les worker nodes

Avantages

- Automatisation et simplification de la tâche d’orchestration
- Evolutivité (scalability) et haute performance: possibilité de déployer plus de conteneurs ou d’ajouter plus de nœuds
- Haute disponibilité (no downtime): les applications restent en ligne, même si certains nœuds sont hors connexion
- Backup et restauration (disaster recovery)

Association Docker et Kubernetes

- Kubernetes est une plateforme d’orchestration alors que Docker est un outil permettant le lancement de conteneur.
- Kubernetes automatise l’exploitation et le control des conteneurs sur une infrastructure distribuée.
  - – Le control des conteneurs devient plus simple et à un niveau plus élevé, sans avoir à gérer chaque nœud ou conteneur de manière individuelle.
- Docker continue de jouer son rôle principal: démarrer et arrêter les conteneurs
  - – Avec Kubernetes les ordres proviennent d'un système automatisé et non plus d’un administrateur qui assigne manuellement les tâches
- Kubernetes permet de travailler avec d’autres outils de conteneurisation mais Docker reste l’option la plus populaire.

Conclusion

- La conteneurisation permet d’isoler des applications avec leur environnement d'exécution complet (contenant tous les fichiers nécessaires à leur fonctionnement).
- Docker et Kubernetes sont des technologies différentes qui fonctionnent bien ensemble pour améliorer la création, le déploiement et la gestion des applications conteneurisées.
- Ensemble, ils permettent de renforcer l’évolutivité et la haute disponibilité des applications.